

CIS520 Project 4: One Program, Three Ways

Group 04: Jackson Seim, Tannyr Singleterry, Jacob Wuthnow

1) Experimental Configuration

All experiments were done on the Kansas State University's Beocat super cluster. We used the slurm batch scheduler. Jobs were constrained to mole class nodes to keep hardware consistent.

2) What We Are Measuring

We aim to show the following performance analysis guidelines:

- Scalability: How does wall time improve as we add threads from 1 to 20?
- Parallel efficiency: Are threads being used effectively, or are they mostly idle?
- RAM usage: Does memory consumptions change with thread count or memory budget?
- I/O vs. computer balance: Is the bottleneck in the file reading or character scanning?
- Implementation comparison: How does pthreads performance compare to MPI for the same work load?

For this first part, the input file sized is fixed at 1.7 GB. We vary thread count and Slurm memory allocation. We repeat each configuration 3 times to obtain standard deviations to assess measurement noise.

3) Software Architecture: Pthreads

This design starts with N threads being crated once at start up and reused for every batch, avoiding repeated pthread_create overhead. The main thread reads lines in batches of up to 4096 using fgets(), signal workers, then waits for all processes to complete before printing results and loading the next batch.

3.1) Software Architecture: MPI

The MPI implementation distributes work across processes using a coordinator/worker pattern. Rank 0 acts as the coordinator. It reads lines form the file and uses MPI_Send to

distribute batches to the workers. Each worker receives its assigned lines, computes the maximum ASCII value, and returns results to rank 0.

3.2) Software Architecture: OpenMP

The OpenMP implementation uses a simple parallel for loop over each batch of lines. The main thread reads lines in batches of up to 4096 using `fgets()` into a flat contiguous buffer, then dispatches computation across threads using a single `pragma omp parallel for` directive. OpenMP's runtime scheduler automatically divides the loop iterations among available threads. After all threads finish, the main thread prints the results in order and loads the next batch.

Compared to `pthread`s, OpenMP requires significantly less synchronization code. There are no mutexes, condition variables, or generation counters. The compiler and runtime handle all thread creation, work distribution, and joining implicitly. Tradeoff is less control over scheduling, but for a simple data parallel loop this is not a concern. Like `pthread`s I/O remains serial on the main thread.

4) Results: Pthreads

Memory Usage

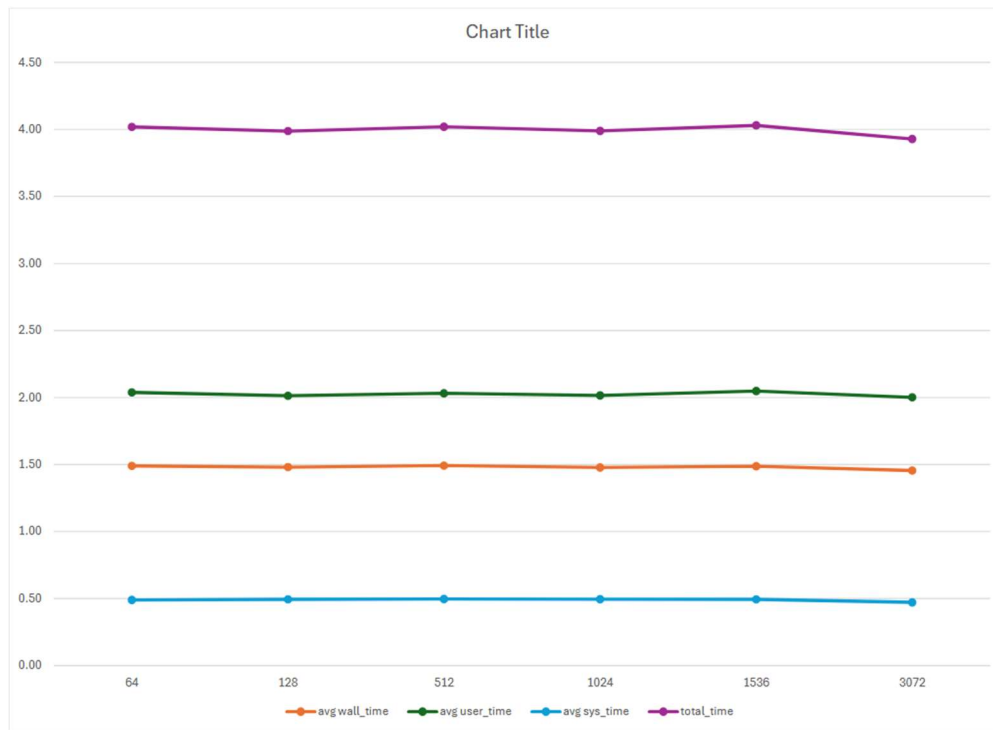


Figure 1: Shows the Amount of Memory Usage per Process.

Thread Count

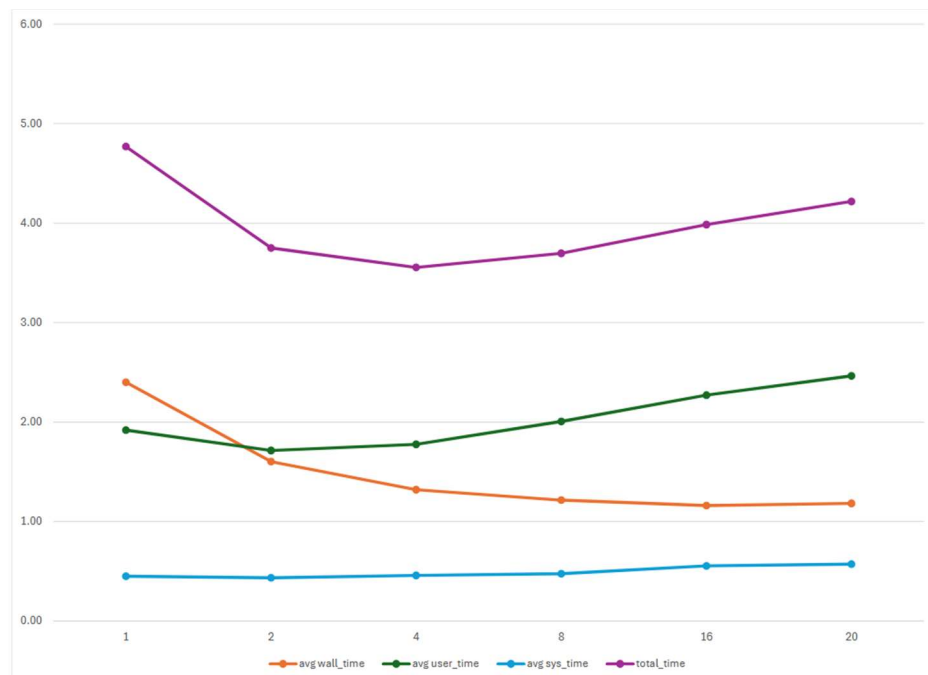


Figure 2: Shows the Number of Threads per Process Over Time

Threads	Mean Wall (s)	Speedup	CPU Efficiency	Peak RSS (MB)
1	2.400	1.00	98.7%	25.6
2	1.602	1.50	67.0%	25.5
4	1.321	1.82	42.3%	25.5
8	1.216	1.97	25.5%	25.5
16	1.161	2.07	15.2%	25.4
20	1.183	2.03	12.8%	25.4

4.1) Results: MPI

Memory Usage

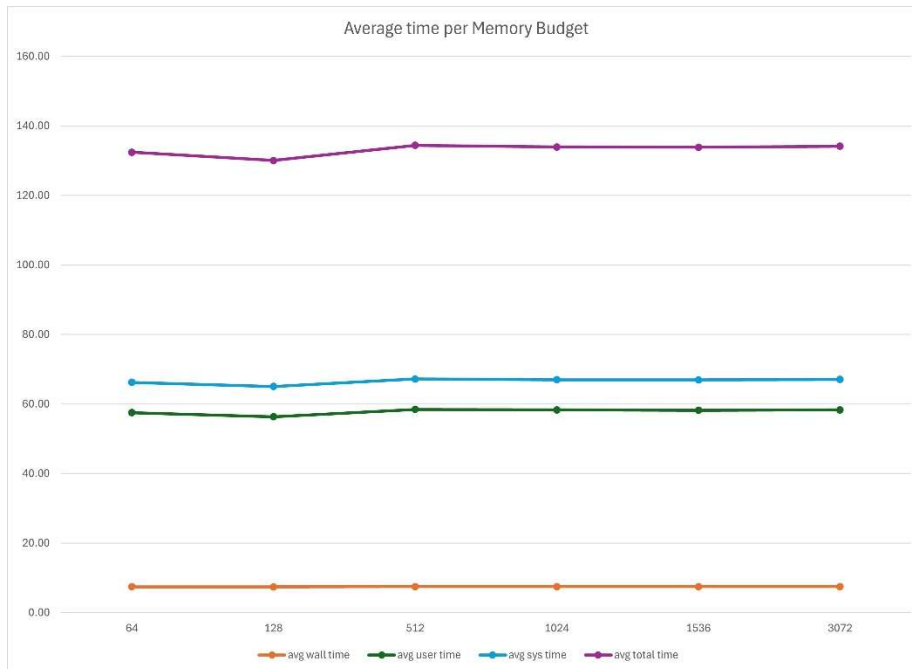


Figure 3: Average Time per Memory Budget per Process.

Like pthreads, memory budget has not measurable effect on MPI wall time. Average wall time stays flat at about 7.6s regardless of budget. However, unlike pthreads, MPI peak RSS does decrease as process count increases, each rank only holds its portion of the data.

Process Count

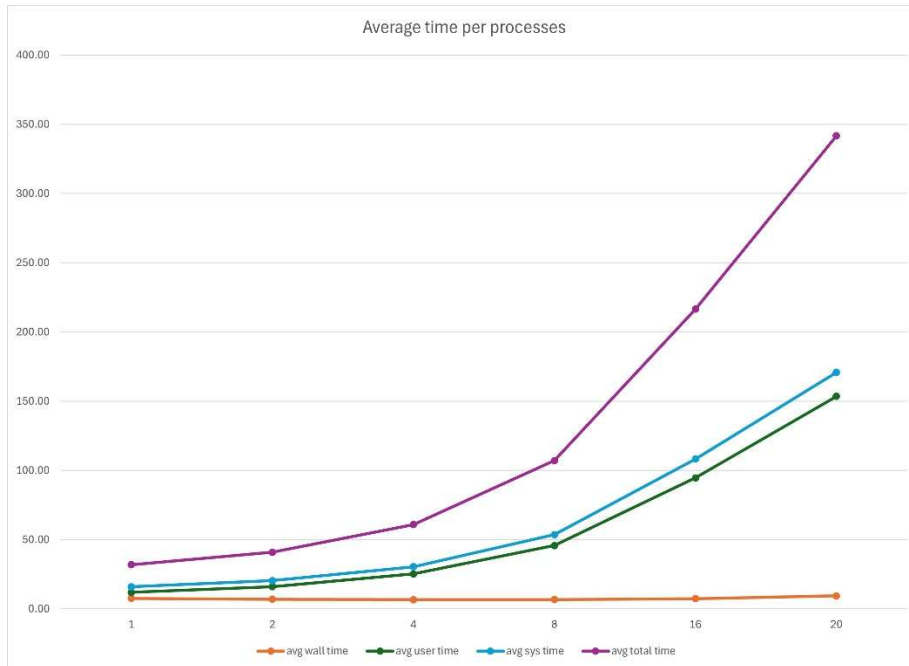


Figure 4: Average Time per Process.

Threads	Mean Wall (s)	Speedup	CPU Efficiency	Peak RSS (MB)
1	7.594	1.00	110.0%	82.1
2	6.941	1.09	97.2%	66.8
4	6.624	1.15	89.8%	59.0
8	6.748	1.13	86.6%	55.6
16	7.387	1.03	85.3%	54.6
20	9.411	0.81	85.7%	54.1

4.2) Results: OpenMP

Memory Usage

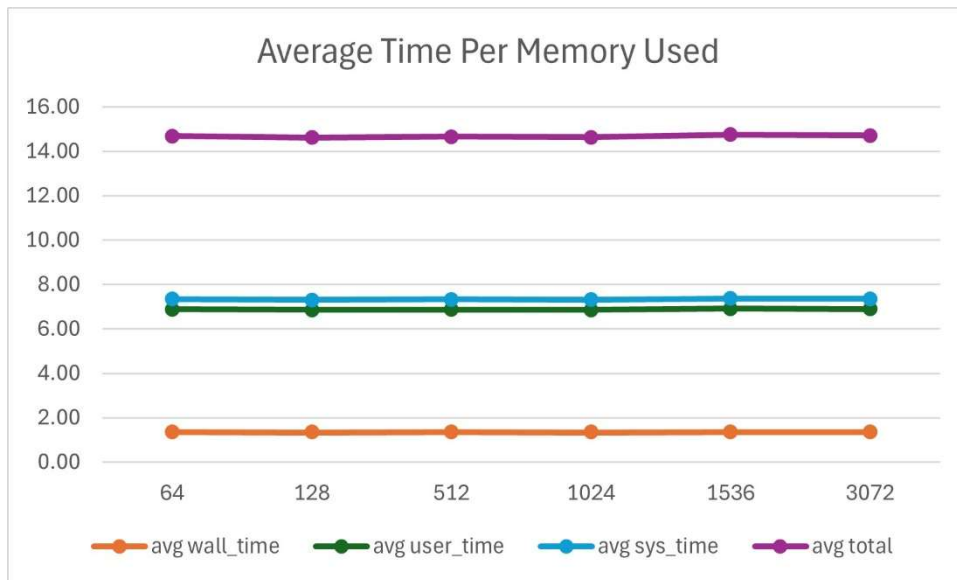


Figure 5: Shows the Amount of Memory Usage Per Process.

Like pthreads, memory budget has no measurable effect on OpenMP wall time or memory usage. Peak RSS stays flat at about 33.8 MB across all thread counts and all memory budgets. This is slightly higher than pthreads (~25.5 MB) due to the single large contiguous buffer allocation ($BATCH_SIZE * MAX_LINE_LEN$ bytes) rather than separate per line allocations.

Thread Count

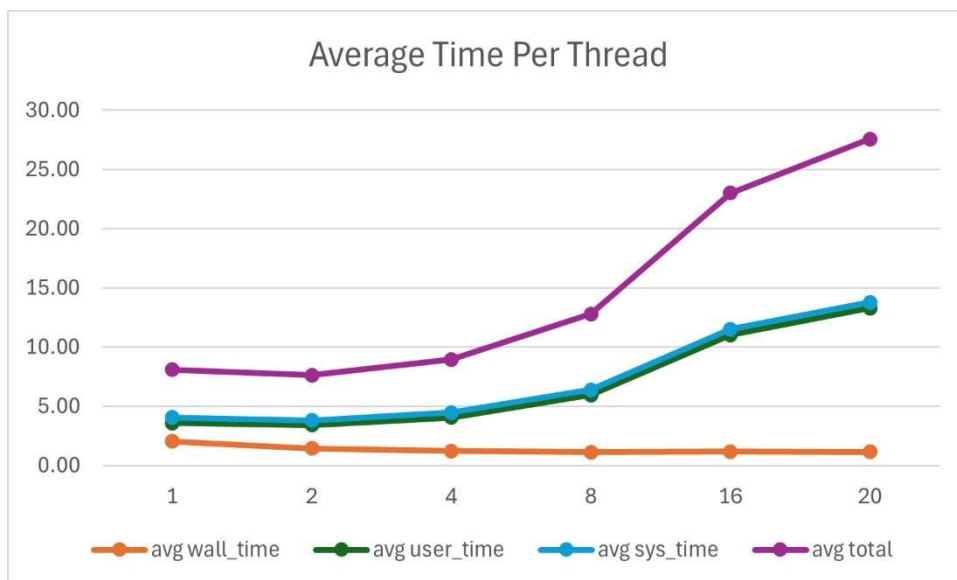


Figure 6: Shows the Number of Threads per Process Over Time.

Threads	Mean Wall (s)	Speedup	CPU Efficiency	Peak RSS (MB)
1	2.030	1.00	99.3%	33.7
2	1.449	1.40	81.8%	33.9
4	1.223	1.66	66.5%	33.9
8	1.136	1.79	58.0%	33.8
16	1.159	1.75	55.8%	33.8
20	1.149	1.77	55.0%	33.8

5) Discussion

Pthreads

The pthreads workload is I/O bound at the application level. The main thread's `fgets()` loop serializes all file reads, creating a ceiling on parallel speedup regardless of how many worker threads are added. Here are major takeaways from the analysis:

- I/O is the dominant bottleneck. Wall time drops from 2.40s at 1 thread to 1.60s at 2 threads, but plateaus at 1.16s beyond 8 threads.
- Diminishing returns beyond 4 threads. Going from 1 to 2 threads saves 0.80s. Going from 8 to 20 threads saves only 0.05s.
- CPU efficiency collapses with thread count. At 20 threads, efficiency falls to 12.8% meaning threads spend 87% of their time blocked on `cond_go` waiting for the main thread to finish reading each back.

MPI

The MPI implementation shows a different and problematic scaling pattern. While CPU efficiency remains high, wall time barely improves and degrades beyond 8 processes. Here are major takeaways from the analysis:

- Wall time barely improves, then worsens. Best wall time is 6.62s at 4 processes. Beyond 4, wall time increases, reaching 9.41s at 20 processes, which is ~24% slower than the 1 process baseline
- Communication overhead dominates. The rank 0 coordinator must sequentially send batches to and receive results from every worker process. As process count grows, this serialized communication loop grows proportionally, overwhelming any parallel benefit from the distributed computation.
- Memory per process decreases with more processes. Peak RSS drops from 82.1 MB at 1 process to 54.1 MB at 20 processes, since each rank only holds its portion of the line buffer.

OpenMP

The OpenMP implantation is the best performing of the three approaches and the simplest to implement. Like pthreads, it is I/O bound, but its lower synchronization overhead gives it a slight edge. Here are major takeaways from the analysis:

- Best wall time of all three. At 20 threads, OpenMP reaches 1.149s compared to 1.183s for pthreads and 9.411s for MPI. The speedup from 1 to 8 threads is similar to pthreads (1.79x and 1.97x respectively), but OpenMP holds slightly better beyond 8 threads.
- Diminishing returns beyond 8 threads. Wall time at 8, 16, and 20 threads is nearly identical (1.136s, 1.159, 1.149s respectively). The I/O bottleneck creates the same ceiling as in pthreads, since the main threads still reads the file serially.
- Higher CPU efficiency than pthreads. At 20 threads, OpenMP CPU efficiency is 55.0% vs 12.8% for pthreads. This is because OpenMP threads are not blocked on condition variables between batches. The runtime handles synchronization at the parallel for barriers more efficiently than the manual cond_go/cond_done mechanism.

6) Comparison of Three Methods and Conclusions

The table below summarizes wall time across all three implementations at each thread/process count:

Count	Pthread Wall (s)	MPI Wall (s)	OpenMP Wall (s)
1	2.400	7.594	2.030
2	1.602	6.941	1.449
4	1.321	6.624	1.223
8	1.216	6.748	1.136
16	1.161	7.387	1.159
20	1.183	9.411	1.149

OpenMP is the clear winner for this workload. Both OpenMP and pthreads scale similarly up to 8 threads, after which all gains plateau due to the serial I/O bottleneck. MPI is fundamentally mismatched to this problem. MPI's communication overhead grows with process count while the single node file read cannot be distributed, causing actual slowdown beyond 4 processes. For a workload dominated by sequential I/O on a single machine, shared memory parallelism is the right tool, and OpenMP achieves that with the lead code complexity.